Configuration Reference

> **Status**: Implementation reference for ADOR 0.5.1. Consolidated list of every `config.toml` setting, its meaning, default value, and examples. Keys are grouped by section.

Profiles & assisted setup (recommended)

You do not have to hand-write `config.toml`. `ador init` runs a guided, per-project wizard that detects prerequisites, prompts explicitly (with examples and validation), and writes a **minimal, validated** config as a named **runtime-project profile** - so one ADOR install serves many projects.

- **Config home (cross-platform):** profiles live under `%APPDATA%\ador` (Windows) / `~/.config/ador` (`$XDG_CONFIG_HOME` respected on Linux), override with the `ADOR_HOME` environment variable. Each profile is `<home>/profiles/<name>/{config.toml, profile.json}`; an `active_profile` pointer names the default.
- **Create:** `ador init [name] [--force] [--answers FILE]` (`--answers` = non-interactive, for CI/servers; keys are the wizard field names). See the onboarding guide (`01_Documentation/Onboarding_New_Project.md`) for the full prompt list and the non-ADOR prerequisite steps.
- **Manage:** `ador profile list|show|use|rename|delete|path` (`list`/`show` include run statistics from the profile's metrics DB; `delete` needs `--force`).
- **Use:** any command resolves its config in this order - explicit `-c FILE` > `--profile NAME` > the active profile > `./config.toml`. Single-config users (no profile) are unaffected. Example: `ador --profile sample-project run --dry-run`.
- **Profile-owned state (default):** a profile's `[state] store_root` points under the profile (`<home>/profiles/<name>/state`), so each project's runs, metrics, and lock stay isolated; the wizard can instead co-locate a `.ador` beside the clone.

Secrets are never written to a profile - only env-key NAMES; tokens/keys stay in a `.env` beside the clone (or CLI host-auth). The rest of this document describes every key the wizard (or you) can set.

Full Example `config.toml`

```
# ----- Bot identity -----
[agent]
name = "ADOR"                 # display/brand: comment tags, MR titles, commit
author
slug = "ador"                 # derived names: .ador/ state dir, branch,
container, marker
email = "ador-bot@example.com"  # commit author email in the container (derived
if empty)

# ----- Git platform -----
[platform]
kind = "gitlab"              # "gitlab" | "github"
repo = "group/subgroup/project"
# The Git-platform CLI (glab / gh) must be installed and authenticated on the
```

```
host.

# ----- State signaling -----
[state]
channel        = "comments"  # "comments" | "labels" | "both"
store_root     = ".ador"      # local authoritative state dir (derived from
slug)

# ----- Polling -----
[polling]
mode                 = "loop"   # "loop" (long-running) | "once"
(cron/scheduler)
poll_interval_seconds = 60       # base wait between polls
poll_jitter_seconds   = 5        # random +/- jitter
poll_backoff          = true     # lengthen interval after Git platform
errors/rate limits
max_concurrent_runs   = 1        # tickets in flight at once
quiet_hours           = []       # e.g. ["22:00-06:00"] windows where polling
pauses

# ----- Readiness filter (selection) -----
[readiness]
require_acceptance_criteria = true
require_priority_label      = true
require_known_component     = true
max_scope_hint              = "small"   # skip tickets larger than this

# ----- Labels -----
[labels.lifecycle]           # orchestrator-managed (used when channel includes
labels)
ready    = "agent-ready"
working  = "agent-working"
clarify  = "agent-clarify"
answered = "agent-answered"
review   = "agent-mr"
failed   = "agent-failed"
done     = "agent-done"

[labels.selection]           # human-managed, read for eligibility/order
priority_high   = "P1"
priority_medium = "P2"
priority_low    = "P3"
type_feature    = "feature"
type_bug        = "bug"

[labels.control]             # human overrides, checked before eligibility
hold    = "agent-hold"
exclude = "no-agent"

[labels.executor]            # optional: pin an executor per ticket
prefix = "agent-exec:"       # e.g. "agent-exec:opencode"

# ----- Executor -----
[executor]
```

```
default        = "opencode"  # "opencode" | "pi" | "codex" | "qwen"
mode           = "thick"     # "thick" (gate-by-gate) | "headless" (single-
shot)
questions_file = "ADOR_QUESTIONS.md"
env_keys       = ["OPENAI_API_KEY", "ANTHROPIC_API_KEY"]  # only these reach
the container
stage_host_auth = false      # pilot-only OAuth bridge for opencode/codex

[executor.opencode]
command = "opencode run --dir {worktree} -m {model} --format json {prompt}"
model   = "<provider/model>"

# To use Pi instead, set `default = "pi"` and add:
# [executor.pi]
# command = "pi -p --mode json --model {model} {prompt}"
# model   = "<provider/model>"
# Pi reads AGENTS.md natively and gets its push/merge guardrails from the
repo's
# .pi agentic-guard extension (see Setup_And_Credentials.md 3).

# To use Codex instead, set `default = "codex"` and add:
# [executor.codex]
# command = "codex exec {resume}--json --model {model} --ignore-user-config --
disable plugins --dangerously-bypass-approvals-and-sandbox {prompt}"
# model   = "<model>"

# To use Qwen Code instead, set `default = "qwen"` and add:
# [executor.qwen]
# command = "qwen -p --model {model} --output-format json {prompt}"
# model   = "<model>"

# ----- Clarification -----
[clarification]
max_rounds     = 3           # ask/answer cycles before escalating to failed
reminder_after = "24h"       # optional single nudge before timeout
answer_timeout = "72h"       # give up and release the ticket after this
timeout_state  = "failed"    # terminal state on timeout ("failed" | "stale")

# ----- Container / isolation -----
[container]
runtime            = "docker"   # "docker" | "podman"
image              = "ador-executor-base:0.2.0"
keep_parked        = true       # keep (stopped) container after parking
parked_ttl_mode    = "polls"    # "seconds" | "polls"
parked_ttl_seconds = 7200       # used when mode = "seconds"
parked_ttl_polls   = 30         # used when mode = "polls"
parked_state       = "stopped"  # "stopped" (fast resume) | "removed" (free
disk)
uv_cache_volume    = "ador-uv-cache"
npm_cache_volume   = "ador-npm-cache"
runtime_env        = { UV_LINK_MODE = "copy" }
run_args           = []         # extra `run` options; rootless podman: ["--
userns=keep-id"]
required_tool_checks = []       # project-image checks, e.g. ["terraform --
```

```
version"]

[container.egress]
mode           = "open"         # "open" | "allowlist-proxy" | "none"
allowlist      = []             # exact/wildcard hosts, e.g. ["api.openai.com",
"*.pythonhosted.org"]
no_proxy       = ["localhost", "127.0.0.1", "::1"]
proxy_image    = "ador-egress-proxy:0.1.0"
proxy_port     = 3128
network_prefix = "ador-egress"

[worktree]
repo_path = "../project" # local clone the run worktrees are created from
(required)
root      = "worktrees"      # host dir holding per-run git worktrees
branch_template = "{slug}/{run_id}"  # run branch name; {slug}/{run_id}/{iid}
placeholders
                                     # e.g. "feature/{slug}-{iid}" for
feature/-prefix push rules
branch_rule = ""             # optional: the repo's branch push-rule regex;
validation
                             # rejects a template the rule would reject, e.g.
                             # '^(feature|bugfix|release|hotfix)\/.*$'

# ----- Run budgets -----
[budgets]
max_iterations         = 20
max_wall_clock_seconds = 3600
max_cost_usd           = 5.0
max_files_touched      = 40

# ----- Delivery -----
[delivery]
target_branch        = "develop"
draft                = true      # open MR/PR as draft; never merge
reviewers            = ["reviewer.one"]
remove_source_branch = true

# ----- Post-MR review loop -----
[review]
trigger               = "label"        # "label" | "comment-marker" | "auto"
revise_label          = "agent-revise" # human applies this to request a
revision
max_review_rounds     = 3
review_reminder_after = "48h"          # optional nudge if the MR sits
undecided

# ----- Reconciliation (external ticket changes) -----
[reconcile]
on_ticket_deleted  = "abort-cleanup"   # abort + remove worktree/container
on_ticket_closed   = "cancel"          # graceful cancel
on_trigger_removed = "pause"           # "pause" | "abort" | "ignore"

# ----- Metrics / statistics (stdlib only; CLI + optional static HTML) -----
```

```toml
[metrics]
enabled          = true
db_path          = ".ador/metrics.db"
export_jsonl     = ".ador/metrics/runs.jsonl"
capture_tokens   = true
capture_resource = true                # docker stats / disk probes
html_out         = ".ador/metrics/stats.html"
```

[agent] - Bot Identity

Key	Meaning	Default
name	Display/brand string; appears in comment tags, MR/PR titles, commit author. ADOR = Agentic Delivery Orchestrator by Randstad.	"ADOR"
slug	Lowercase token that derives the state dir (.<slug>/), branch prefix, container name, questions file, and comment marker. Derived from name if empty.	"ador"
email	Git commit author email inside the executor container (name is the author name). Derived as <slug>-bot@local if empty.	(derived)

[platform] - Git Platform Selection

Key	Meaning	Default
kind	Which Git-platform adapter to use.	"gitlab"
repo	Repository path/slug passed to the Git-platform CLI.	(required)

Note: glab (GitLab) or gh (GitHub) must be installed and authenticated on the host. No separate bot account is required (identity is marker-based).

[state] - State Signaling

Key	Meaning	Default
channel	How run state is reflected on the ticket: comments, labels, or both. Authoritative state always lives locally. In comment mode ADOR keeps one status comment and edits it in place.	"comments"
store_root	Local authoritative state directory.	".ador"

Comment Marker Format

ADOR comment identity is marker-based, not author-account-based. Every ADOR comment has a visible tag plus a hidden HTML comment marker.

Visible status line example:

```
[ADOR] status: working - stage check - run <run-id> - updated <utc-timestamp>
```

Hidden marker format:

```
<!-- <agent-slug>:run=<run-id> kind=<kind> state=<state> stage=<stage> -->
```

Default example:

```
<!-- ador:run=gitlab-example-project-52 kind=status state=working stage=check -
->
```

The marker lets ADOR distinguish its own comments from human comments even when the same human account runs the platform CLI. Comments without the matching marker are treated as human-authored. This matters for clarification resume: when ADOR parks a run, a normal human comment can answer the question because it does not carry the ADOR marker.

Do not manually edit or remove marker lines from ADOR comments. The marker grammar is intentionally not configurable; it is created and parsed by the identity component. The configurable part is the agent identity (`name` and `slug`), which changes the visible tag and the marker prefix.

`[polling]` - Polling

Key	Meaning	Default
`mode`	`loop` = long-running service; `once` = single pass for cron/Task Scheduler.	`"loop"`
`poll_interval_seconds`	Base wait between polls (configurable period).	`60`
`poll_jitter_seconds`	Random jitter added to the interval.	`5`
`poll_backoff`	Lengthen the interval after Git platform errors / rate limits.	`true`
`max_concurrent_runs`	Tickets allowed in flight at once.	`1`
`quiet_hours`	Windows where polling pauses.	`[]`

`[readiness]` - Readiness Filter

Key	Meaning	Default
`require_acceptance_criteria`	Skip tickets with no acceptance criteria.	`true`
`require_priority_label`	Require a P-label to be eligible.	`true`
`require_known_component`	Require the ticket to map to a known component.	`true`

Key	Meaning	Default
`max_scope_hint`	Skip tickets estimated larger than this.	`"small"`

`[labels.*]` - Labels

Group	Meaning	Managed by
`labels.lifecycle`	Run-state labels (`ready`/`working`/`clarify`/`answered`/`review`/`failed`/`done`). Used only when `state.channel` includes labels.	orchestrator
`labels.selection`	Eligibility/priority/type labels read at pickup (`P1`..`P3`, `feature`, `bug`).	humans
`labels.control`	Overrides checked before eligibility (`hold`, `exclude`).	humans
`labels.executor`	Optional prefix to pin an executor per ticket (`agent-exec:opencode`).	humans

Defaults are listed in the example above. Every label string is configurable so teams can rename without code changes; the orchestrator refers to labels by role.

`[executor]` - Executor

Key	Meaning	Default
`default`	Which executor to launch: `opencode`, `pi`, `codex`, or `qwen`. The `[executor.<name>]` table must match. `opencode` is the default and live-proven; the other launchers are wired behind the same factory and require their project guard/auth prerequisites.	`"opencode"`
`mode`	`thick` = orchestrator drives gate-by-gate; `headless` = single-shot.	`"thick"`
`questions_file`	File the executor writes clarification questions to.	`"ADOR_QUESTIONS.md"`
`env_keys`	The only environment variables (from host env / `.env`) injected into the executor container.	`["OPENAI_API_KEY", "ANTHROPIC_API_KEY"]`
`stage_host_auth`	Pilot-only: copy host auth into the run container at provision. Implemented for opencode (`~/.local/share/opencode/auth.json`) and Codex (`~/.codex/auth.json` staged into an ADOR-owned `CODEX_HOME`).	`false`
`[executor.<name>].command`	Command template for that executor; `{prompt}` (shell-quoted), `{model}`, and `{worktree}` (container path `/ador/work`) are substituted. opencode/pi continue with `--session <id>`; Codex uses the `{resume}` placeholder for `codex exec resume --`	per executor

Key	Meaning	Default
	`last`; Qwen is stateless per gate. Safety validation rejects opencode `--auto`, Codex commands missing non-interactive/user-config guards, and Qwen yolo mode.	
`[executor.<name>].model`	Model/provider string for that executor.	(required)

`[clarification]` - Clarification

Key	Meaning	Default
`max_rounds`	Ask/answer cycles before escalating the ticket to `failed`.	`3`
`reminder_after`	Optional single nudge (mentions assignee) before the timeout, measured from `parked_at`.	`"24h"`
`answer_timeout`	Give up and release the ticket if no human reply within this window.	`"72h"`
`timeout_state`	Terminal state applied on timeout (`failed` or a dedicated `stale`).	`"failed"`

`[container]` - Container / Isolation

Key	Meaning	Default
`runtime`	Container runtime command: `docker` / `podman`, optionally wrapped, e.g. `"wsl -d Ubuntu-24.04 -- docker"` for Docker Engine inside WSL2 (ADOR then translates mount paths and propagates env via WSLENV).	`"docker"`
`image`	Container image selected by the runtime-project profile. The generic ADOR default is executor-only; project images extend it with project dependencies.	`"ador-executor-base:0.2.0"`
`keep_parked`	Keep (stopped) the container after parking instead of removing.	`true`
`parked_ttl_mode`	Grace period unit: `seconds` or `polls`.	`"polls"`
`parked_ttl_seconds`	Reap idle parked container after N seconds (mode=seconds).	`7200`
`parked_ttl_polls`	Reap idle parked container after N poll cycles (mode=polls).	`30`
`parked_state`	On park: `stopped` (fast resume) or `removed` (free disk now).	`"stopped"`
`uv_cache_volume`	Persistent uv cache volume for fast `uv sync`.	`"ador-uv-cache"`

Key	Meaning	Default
`npm_cache_volume`	Persistent npm cache volume for fast `npm ci`.	`"ador-npm-cache"`
`runtime_env`	Non-secret environment injected at container creation and every `exec`. Use it for portable tool behavior such as `UV_LINK_MODE=copy` or a project-specific `UV_PROJECT_ENVIRONMENT`; ADOR rejects overrides of managed isolation/proxy keys (`HOME`, `GIT_*`, `CODEX_HOME`, `HTTP_PROXY`, `HTTPS_PROXY`, `ALL_PROXY`, `NO_PROXY`, and lowercase variants).	`{ UV_LINK_MODE = "copy" }`
`run_args`	Extra option tokens for `<runtime> run` (one option per entry, no spaces; validated). ROOTLESS PODMAN REQUIRES `["--userns=keep-id"]` (verified 2026-07-04: without it container uid 1000 maps to a subuid that cannot write the bind-mounted worktree). Also the seam for future resource limits (`"--memory=2g"`).	`[]`
`required_tool_checks`	Shell-free command strings run inside the provisioned container before any executor gate starts. Use this for runtime-project images, for example `["make --version", "shellcheck --version", "terraform --version", "tflint --version"]`.	`[]`

`[container.egress]` - Executor Network Egress

Key	Meaning	Default
`mode`	`open` preserves supervised-pilot behavior; `none` starts the executor with `--network none`; `allowlist-proxy` gives the executor no direct internet path and injects HTTP/HTTPS proxy variables pointing at an ADOR-managed per-run proxy.	`"open"`
`allowlist`	Exact or wildcard hostnames allowed through the proxy, for example `["gitlab.example.com", "api.openai.com", "*.pythonhosted.org"]`. Schemes, paths, whitespace, shell characters, empty entries, and invalid hostnames are rejected. Required and non-empty for `allowlist-proxy`.	`[]`
`no_proxy`	Hostnames bypassed by proxy-aware tools. Loopback defaults should usually remain.	`["localhost", "127.0.0.1", "::1"]`
`proxy_image`	Generic Squid proxy image built from `docker/Dockerfile.egress-proxy`; no project hosts are baked into the image.	`"ador-egress-proxy:0.1.0"`

Key	Meaning	Default
`proxy_port`	Port exposed by the per-run proxy container and used in injected proxy variables.	`3128`
`network_prefix`	Prefix for per-run internal networks, followed by the safe executor container name.	`"ador-egress"`

Unattended deployment guardrail: systemd and Task Scheduler installers refuse `mode = "open"` unless `--allow-open-egress` / `-AllowOpenEgress` is passed. Manual supervised `ador run --once` remains allowed with open egress.

`[worktree]` - Worktree

Key	Meaning	Default
`repo_path`	Local clone of the target repository that per-run worktrees are created from; mounted (with the worktree) into the executor container.	(required)
`root`	Host directory holding per-run git worktrees (`<root>/<run-id>`).	`"worktrees"`
`branch_template`	Run branch name; `{slug}`, `{run_id}`, `{iid}` placeholders (must keep `{run_id}` or `{iid}` for uniqueness). Set e.g. `"feature/{slug}-{iid}"` for repos whose push rules require a `feature/` prefix.	`"{slug}/{run_id}"`
`branch_rule`	Optional regex mirroring the repo's branch-name push rule, for example `^(feature|bugfix|release|hotfix)\/.*$`; validation rejects a template the rule would reject.	`""`

`[budgets]` - Run Budgets

Key	Meaning	Default
`max_iterations`	Max **executor launches** per run before failing closed. What counts: one launch per gate (6 for a clean pass) plus one per fix launch (merge-conflict resolution or a failing-gate fix during delivery) and per revise round. Clean delivery (no conflicts, gates green) launches the executor **zero** extra times - merge/preflight/push/MR are deterministic orchestrator actions. Sizing: `6` = clean pass only, zero fix headroom (crawl pilots); `12-15` = supervised operation with fix headroom; `20` (default) = unattended headroom. Tripping the budget never loses work: the run fails with the reason journaled and posted, and `ador retry` re-arms the counters and continues from the stored stage.	`20`

Key	Meaning	Default
`max_wall_clock_seconds`	Max wall-clock of **active work** per run; the clock restarts on `ador retry` and on resume-from-park (parked waiting time is bounded by `[clarification] answer_timeout` instead).	`3600`
`max_cost_usd`	Max model spend per run.	`5.0`
`max_files_touched`	Max files a run may change.	`40`

`[delivery]` - Delivery Boundary

Key	Meaning	Default
`target_branch`	Branch the MR/PR targets.	`"develop"`
`draft`	Open as draft; the orchestrator never merges.	`true`
`reviewers`	Human reviewers assigned to the MR/PR.	(required)
`remove_source_branch`	Remove the source branch on merge.	`true`

`[review]` - Post-MR Review Loop

Key	Meaning	Default
`trigger`	How a revise cycle starts: `label` (explicit; implemented), `comment-marker`, or `auto` (both currently treated as `label`).	`"label"`
`revise_label`	Label a reviewer applies to request an agent revision.	`"agent-revise"`
`max_review_rounds`	Revise cycles before escalating the run to `failed`.	`3`
`review_reminder_after`	Optional nudge if the MR sits undecided.	`"48h"`

Note: approval and merge are always human actions; the revise cycle only updates the branch (the MR stays a draft).

`[reconcile]` - Reconciliation

Key	Meaning	Default
`on_ticket_deleted`	Action when a run's ticket is deleted.	`"abort-cleanup"`
`on_ticket_closed`	Action when a run's ticket is closed by a human.	`"cancel"`
`on_trigger_removed`	Action when the trigger label is removed / `agent-hold` added mid-run (`pause`/`abort`/`ignore`).	`"pause"`

Manual overrides (`ador cancel/delete/reset/retry/cleanup`, `ador pause/resume`) are CLI commands, not config; see design doc section 8c.

`[metrics]` - Statistics

Key	Meaning	Default
`enabled`	Turn statistics capture on/off.	`true`
`db_path`	SQLite database of run summaries (stdlib `sqlite3`).	`".ador/metrics.db"`
`export_jsonl`	Mirrored append-only export of run summaries.	`".ador/metrics/runs.jsonl"`
`capture_tokens`	Record executor token usage per run/gate.	`true`
`capture_resource`	Probe container/disk resource stats.	`true`
`html_out`	Output path for `ador stats --html` static dashboard.	`".ador/metrics/stats.html"`

Access is CLI-first (`ador stats`, `ador report`, `ador export`) plus the optional static HTML; there is no Prometheus endpoint or hosted service.

Notes

- Every value above is a proposed default and may be tuned per environment.
- Related but non-`config.toml` inputs (kept as environment variables / secrets, not in this file): the Git-platform token used by `glab` / `gh`, and the executor's model-provider credentials.
- Cross-references: polling (design doc section 5a), labels (5b), identity and signaling (5c), run identity and local state (8a), container TTL (8b), clarification (9), isolation and dependencies (10a), delivery (11).